

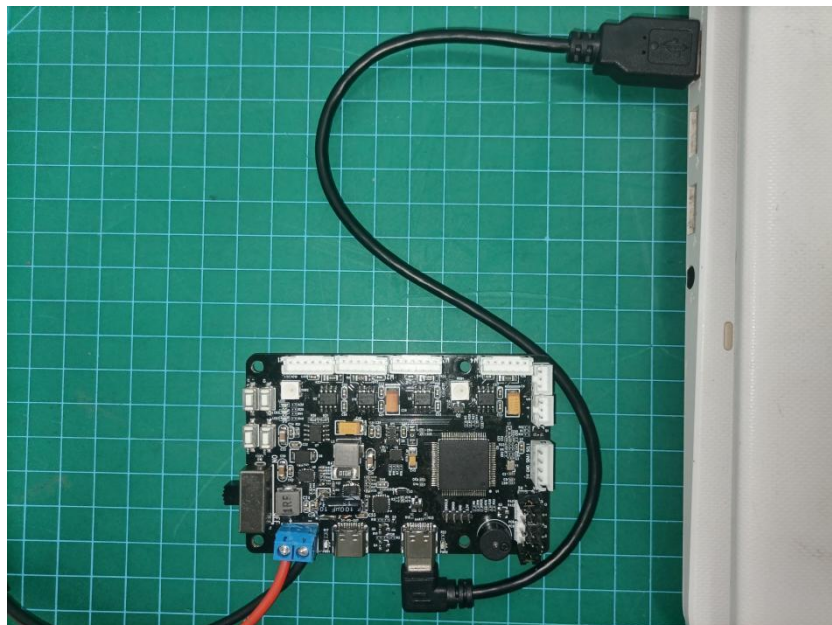
Lesson 6 RGB LED Display Source Code Analysis

1. Program Description

In this section, we will learn how to use the PA5 and PA7 pins on the STM32F407 as the master device of SPI protocol. By outputting data to the input pin (SPI1_MOSI) and clock line (SPI1_SCK) of the two WS2812b series RGB LEDs on the development board, the color of the RGB LEDs can be controlled. The RGB LEDs will emit mixed light of red, green, and blue in different ratios, based on the data they receive.

2. Hardware Connection

Connect the RRCLite controller to the PC via USB cable through serial port 1. The hardware connection diagram is as follows:



3. Program Download

The example for this section is "RRCLite_RGB".

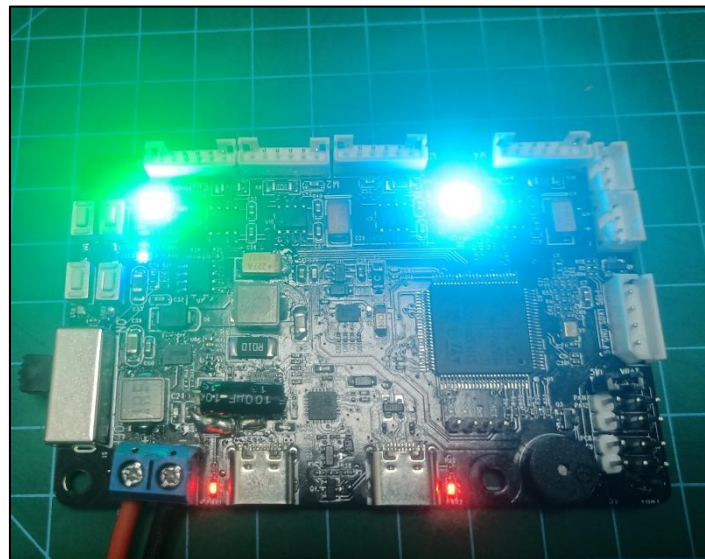
4. Program Outcome

After downloading the program, press the RESET button to make two RGB lights light up with different color ratios.

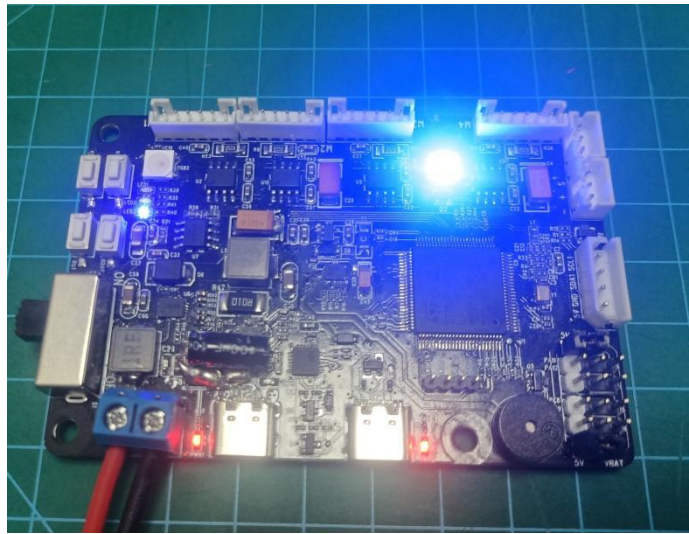
Specifically, after powering up, RGB light 1 displays blue, and RGB light 2 does not light up. After 5 seconds, RGB light 2 turns into green, and RGB light 1 does not light up. Then, after another 5 seconds, both RGB lights turn into red.

After 5 seconds, RGB light 1 returns to its original state, displaying blue. RGB light 2 does not light up. This cycle repeats continuously.

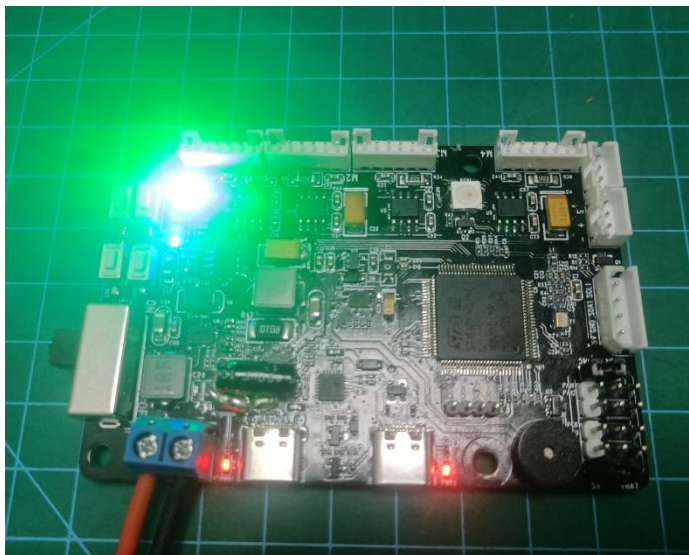
Note: The MCU can control multiple RGB lights connected in series through an SPI interface. The program can also be modified to achieve separate control of one or more of these RGB lights.



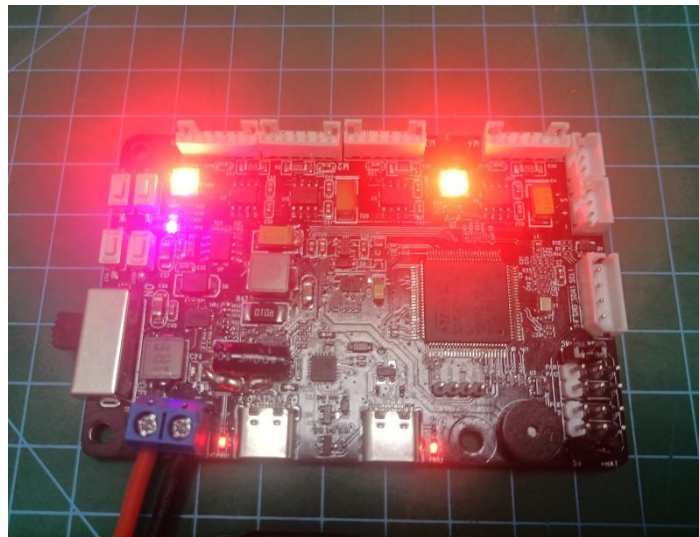
(During initialization)



(RGB light 0 displays blue)



(RGB light 1 displays green)



(RGB lights all display red)

5. Pin Configuration

Use STM32CubeMX software to view the pin information. The microcontroller communicates with the IMU using SPI. According to the following figure, the corresponding pins for MOSI and SCK functions in the SPI protocol are PA6 and PA7 respectively. Since the RGB light serves as a slave device and does not need to report any information to the MCU, MISO and other functions are simplified to save pin resources.

PA7	SPI1_MOSI
PA8	BUZZER
PA6	SPI1_SCK

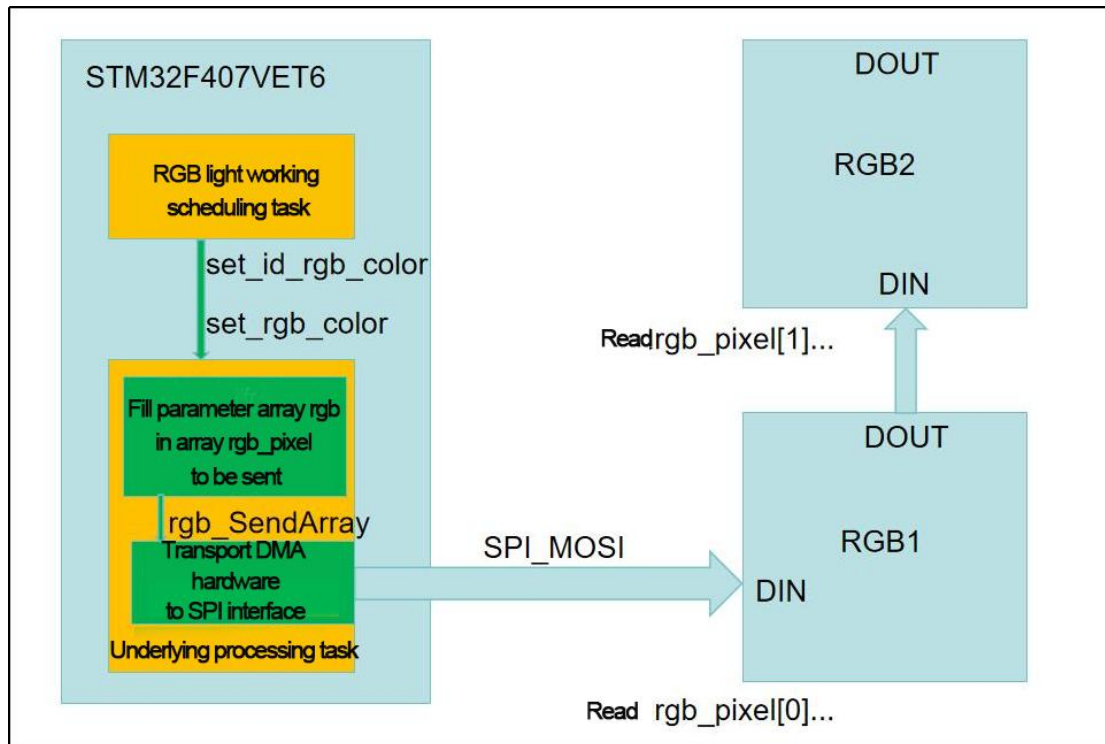
6. Source Code Analysis

The source code for this program is under the same directory as this lesson.

The main program framework is designed as follows:

The main analysis of the RGB light working scheduling task is based on the Hiwonder/System/app.c file, and the underlying processing task is based on

the Hiwonder/Peripherals/rgb_spi.c file.



It can be divided into two parts: RGB light working scheduling task, and underlying processing task.

6.1 RGB Light Working Scheduling Task

The task is located in the `app.c` file under the `Hiwonder/System` directory, inside the `app_task_entry(void *argument)` function. This task function is dedicated to user secondary development and implementation of DIY functions, as follows:


```
void app_task_entry(void *argument)
{
    extern osTimerId_t led_timerHandle;
    extern osTimerId_t buzzer_timerHandle;
    extern osTimerId_t button_timerHandle;
    extern osTimerId_t battery_check_timerHandle;
    extern osMessageQueueId_t moving_ctrl_queueHandle;

    leds_init();
    motors_init();
    pwm_servos_init();
    serial_servo_init();
    buzzers_init();
    buttons_init();
    WS2812b_Configuration(); ①

    button_register_callback(buttons[0], button_event_callback);
    button_register_callback(buttons[1], button_event_callback);

    osTimerStart(led_timerHandle, LED_TASK_PERIOD);
    osTimerStart(buzzer_timerHandle, BUZZER_TASK_PERIOD);
    osTimerStart(button_timerHandle, BUTTON_TASK_PERIOD);
    osTimerStart(battery_check_timerHandle, BATTERY_TASK_PERIOD);
    packet_handle_init();

    // osDelay(50);
    // char msg = '\0';
    // uint8_t msg_prio;
    // osMessageQueueReset(moving_ctrl_queueHandle);
    // chassis_init();
    // set_chassis_type(CHASSIS_TYPE_TT4WD);
    // led_on(leds[0]);
    // led_off(leds[1]);
    // led_flash(leds[2], 100, 1000, 0);

    uint8_t rgb_all[6] = {0};
    uint8_t rgb[3] = {0}; ②
}
```

At ①, the initialization of RGB light configuration information is implemented. The function is responsible for clearing the part of the data array to be sent from SPI to configure the RGB light, and simultaneously writing the fixed content format at the end of the data array.

At ②, two arrays, rgb (used to control a single RGB light) and rgb_all (used to control two RGB lights simultaneously), are created. They can be written in this task function.



In the loop section of the program, at ①, the array `rgb` first sets the part controlling the green (0-7 bit) to 255 (or any value between 0-255; the larger the value, the deeper the displayed green, and the same applies to other colors). Then, the `rgb` is passed to RGB light 1, which has an ID of 0 because the configuration array index starts from 0. The ID of RGB light 2 is 1.

After clearing the array `rgb`, it is passed to RGB light 2.

This can achieve the effect: RGB light 1 turns blue and RGB light 2 does not light up after the program is enabled.

After a 5-seconds delay, the second segment of the program starts to be executed, which is similar to the previous segment and does not require further analysis.

In the third segment of the program, the `rgb_all` array is used to control two RGB lights simultaneously. It is achieved by setting the configuration information of each RGB light's red component to 255 in `rgb_all`, and then passing it to both RGB lights. This allows both RGB lights 1 and 2 to light up red simultaneously.

When the `set_id_rgb_color` and `set_rgb_color` functions are executed, the underlying processing task starts to work.

Introduction to WS2812b: Each RGB LED has a chip inside, which controls its illumination by receiving specific timed data. When sending data, 24 bytes of RGB data need to be sent in a specific timed sequence, with the high bits first and the low bits last, and the format is GRB. The color information in the configuration array is arranged in the order of green-red-blue.

6.2 Underlying Processing Task

```
#define Pixel_S1_NUM 2 //LED bead RGB quantity
#define CODE0 0xC0 // 0 code, the sending time 1100 0000 should be adjusted appropriately according to different SCKs
#define CODE1 0xFC // 1 code, sent at 1111 1100
```

```
static uint8_t pixelBuffer[Pixel_S1_NUM+1][24]; //LED bead
//uint8_t *pixelBuffer;
```

At the low level, a pixelBuffer array is created to receive the configuration information sent by the RGB LED scheduling task. It should be noted that the sent array data cannot be used directly.

The original array data, which allocates 3 unsigned char spaces for each RGB LED to store the green-red-blue color configuration. Each color of the RGB LED is represented by 8 bits. It needs to be converted into 0s and 1s that can be recognized by the LED when transmitted at the low level.

The configuration array sent by the RGB LED scheduling task needs to be broken down into each bit of each element and converted into the corresponding 0s and 1s, which are then written into the pixelBuffer array in order.

After sending the data, a reset signal is needed to indicate the end of transmission, which is a low-level signal lasting 280us. In this program, the method of sending a 0 element of the pixelBuffer array size is used to implement the reset code.

Therefore, the size of the pixelBuffer array can be determined as long as the total number of RGB LEDs is known.


```
void WS2812b_Configuration(void)
{
    memset(pixelBuffer, CODE0, Pixel_S1_NUM*24);
    memset(pixelBuffer[Pixel_S1_NUM], 0x00, 24);
}
```

The above figure shows the initialization method of the pixelBuffer data array at the low level. The data elements that control each RGB LED are initialized separately from the last element that represents the reset code. Please note the difference between 0s and 0 to avoid misuse.

Note: this function must be called before the function call in the following text (otherwise the space for storing the reset code is not properly defined), and it will not be recognized correctly by the device during transmission.

```
void rgb_SendArray(void){
    HAL_SPI_Transmit_DMA(&hspi1, (uint8_t*)pixelBuffer, (Pixel_S1_NUM+1)*24);
}
```

The above figure shows the method of transmitting the low-level data stored in the MCU to the RGB LED, which is transmitted to the MOSI1 interface of SPI1 through DMA hardware to reduce the transmission burden on the CPU.

It should be noted that this function has been integrated into the function in the following text, so the RGB LED scheduling task does not need to call this function separately.

```
void set_id_rgb_color(uint8_t id, uint8_t* rgb)
{
    if(id >= Pixel_S1_NUM) //The RGB light numbers are 0 and 1
    {
        return;
    }
    int i = 0;
    for(i=0;i<=7;i++){
        pixelBuffer[id][i]= ( (rgb[0] & (1 << (7 -i))) )? (CODE1):CODE0 );
    }
    for(i=8;i<=15;i++){
        pixelBuffer[id][i]= ( (rgb[1] & (1 << (15-i))) )? (CODE1):CODE0 );
    }
    for(i=16;i<=23;i++){
        pixelBuffer[id][i]= ( (rgb[2] & (1 << (23-i))) )? (CODE1):CODE0 );
    }
    rgb_SendArray();
}
```

The above figure shows the method of controlling a single RGB LED, which involves obtaining the ID of the RGB LED and its configuration data. The space called by the array parameter is "3*sizeof(uint8_t)".

Then, the corresponding space allocated to the RGB LED with that ID is

written into the low-level data array pixelBuffer, while the configuration information of other RGB LEDs remains unchanged (keeping their previous color state). Finally, the data is transmitted from the low-level data array to the RGB LED.

```
void set_rgb_color(uint8_t* rgb)
{
    int i = 0;
    for(i=0;i<=7;i++){
        pixelBuffer[0][i]= ( (rgb[0] & (1 << (7 -i))) )? (CODE1):CODE0 );
    }
    for(i=8;i<=15;i++){
        pixelBuffer[0][i]= ( (rgb[1] & (1 << (15-i))) )? (CODE1):CODE0 );
    }
    for(i=16;i<=23;i++){
        pixelBuffer[0][i]= ( (rgb[2] & (1 << (23-i))) )? (CODE1):CODE0 );
    }
    for(i=0;i<=7;i++){
        pixelBuffer[1][i]= ( (rgb[3] & (1 << (7 -i))) )? (CODE1):CODE0 );
    }
    for(i=8;i<=15;i++){
        pixelBuffer[1][i]= ( (rgb[4] & (1 << (15-i))) )? (CODE1):CODE0 );
    }
    for(i=16;i<=23;i++){
        pixelBuffer[1][i]= ( (rgb[5] & (1 << (23-i))) )? (CODE1):CODE0 );
    }
    rgb_SendArray();
}
```

The above figure shows the method of controlling both RGB LEDs simultaneously. The space called by the array parameter is “2*3*sizeof(uint8_t)”.